

Promises

A **Promise** is an object that represents the **future result of an asynchronous operation**.

👉 Think of it like ordering food online:

- You **place order** → Pending
- Food arrives → Fulfilled
- Restaurant cancels → Rejected

So a Promise is basically:

“I don't have the result now, but I will give it to you later.”

Why Promises Exist (Problem They Solve)

Before Promises → we used **callbacks**.

✗ Callback Hell:

```
login(user, function() {  
  getProfile(function() {  
    getPosts(function() {  
      getComments(function() {  
        // nested mess ☹️  
      });  
    });  
  });  
});
```

Problems:

- Hard to read
- Hard to debug
- Error handling is messy
- Not scalable

✓ Promises fix this with **structured async flow**.

◆ Promise States (Life Cycle)

Every Promise has **3 states**:

State	Meaning
Pending	Still working
Fulfilled	Success
Rejected	Failed

```
const myPromise = new Promise((resolve, reject) => {  
  let success = true;  
  
  if(success){  
    resolve("Data received");  
  } else {  
    reject("Error occurred");  
  }  
});
```

Important:

- resolve() → success

- `reject()` → failure

The function inside Promise runs **immediately**.

Consuming a Promise

We use:

- `.then()` → success
- `.catch()` → error
- `.finally()` → always runs

`myPromise`

```
.then(data => {  
  console.log(data);  
})  
.catch(err => {  
  console.log(err);  
})  
.finally(() => {  
  console.log("Done");  
});
```

Real Example (Simulating API Call)

```
function fetchUser(){  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve({name:"Prajwal", role:"Developer"});  
    },2000);  
  });  
}
```

```
}
```

```
fetchUser()  
  .then(user => console.log(user))  
  .catch(err => console.log(err));
```

Promise Chaining (Most Important Concept)

Instead of nesting → we chain.

```
getUser()  
  .then(user => getOrders(user.id))  
  .then(orders => getPayment(orders))  
  .then(payment => console.log(payment))  
  .catch(err => console.log(err));
```

Each `.then()` returns **a new Promise**.

This is what removes callback hell.

◆ Returning Values vs Returning Promises

Case 1: Returning value

```
.then(data => {  
  return data * 2;  
})
```

JS automatically wraps it inside Promise.

Case 2: Returning Promise

```
.then(data => {  
  return fetchAnotherData();  
})
```

```
})
```

Chain **waits automatically**.

This is why Promises feel synchronous.

Promise Error Propagation (Interview Favourite)

Error anywhere → goes to nearest `.catch()`.

```
getUser()
  .then(user => {
    throw new Error("Something broke");
  })
  .then(() => {})
  .catch(err => console.log(err.message));
```

You don't need multiple catches.

Promise.all() (Parallel Execution)

Runs multiple async tasks together.

```
Promise.all([fetchUsers(), fetchPosts(), fetchComments()])
  .then(([users, posts, comments]) => {
    console.log(users, posts, comments);
  });
```

✓ Faster → runs in parallel return array of fulfilled promises

✗ If ONE fails → whole fails.

Promise.allSettled()

Waits for all — even if some fail.

Promise.allSettled([p1,p2,p3])

Returns status of each.

Promise.race()

First one to finish wins.

Useful for:

- timeout handling
- fastest API response

◇ Promise.any()

First **successful** one wins.

1. What is a Promise?

Answer:

A Promise is an object representing the **eventual completion or failure** of an asynchronous operation.

It helps manage async code **without callback hell**.

2. What is the difference between resolve() and reject()?

- resolve(value) → marks Promise as **success**
- reject(error) → marks Promise as **failed**

resolve(data); // goes to .then()

reject(error); // goes to .catch()

3. What does `.then()` return?

Important Question (frequently asked).

`.then()` **always** returns a new **Promise**.

This allows chaining:

4. What happens if you return a value inside `.then()`?

It automatically becomes:

```
return Promise.resolve(value);
```